

WiFi Sense — Challenge 1: Smart Home

Josip — Fontys ICT, Semester 1, Block 1

Introduction

For the Smart Home challenge I'm building **WiFi Sense**: a presence-detection system for my studio room (~20 m²). A small radar sensor tracks where people are in the room and shows them as moving dots on a live, top-down "radar" dashboard. Because it uses radar instead of a camera, it works in the dark and records no images — it can tell *that* and *where* someone is, but not *who* or what they look like.

The problem it solves for me personally: I live alone in a student studio and want to know if anyone enters my room while I'm away, without hanging a camera in my own living space. Later versions should let me open the dashboard on my phone from anywhere and send me a notification when presence is detected while I'm out.

Stakeholders: me (user and developer), my teachers(testers and feedback), and in a broader sense anyone who wants home monitoring without cameras.

Main question

How can I design and build a system that reliably detects and tracks people in a room without using a camera, and shows this live on a dashboard?

How the system works

radar sensors → ESP32 microcontroller → Python server → web dashboard

Component	What it does
HLK-LD2450 radar	tracks up to 3 people: x/y position + speed, 10x/second
HLK-LD2412 radar	detects motionless people (breathing micro-motion) — until it's added, the server covers the LD2450's still-person blind spot in software
AM312 PIR	classic infrared motion sensor, used as a cross-check against false alarms
ESP32	reads the sensors, sends the data over WiFi
Python server on my PC	converts sensor data to room coordinates, serves the dashboard
Web dashboard	top-down room map with live person dots

There is also a research side: I'm experimenting with sensing people through **WiFi signals themselves** (Channel State Information — a person moving through a room disturbs the WiFi signals bouncing around it, and that disturbance can be measured with two cheap ESP32 boards). It's the same physics as radar but much harder to get answers from, so I use the radar system as the "correct answer" to compare the WiFi experiment against.

Target audience

The first user is me: a student living alone in a studio who wants to know if anyone enters while I'm away, without putting a camera in the room I sleep in. But the same system fits anyone who wants home monitoring where a camera feels wrong:

- **Students and renters** in shared housing — monitoring your own

room without filming housemates who walk past.

- **Privacy-conscious households** — presence detection in bedrooms and bathrooms, where cameras are unacceptable.
- **Elderly care** — knowing a person is present and moving (or motionless for too long) without watching them, a growing real market for this exact sensor technology.

Why this instead of a camera or a cheap motion sensor?

- **No images exist at all.** A camera's privacy depends on trusting the software; radar physically cannot take a picture. It sees positions, not faces.
- **Works in complete darkness** — and through thin obstacles like a curtain.
- **Detects people sitting still.** The classic cheap solution (a PIR motion sensor, like in hallway lights) goes blind the moment you stop moving. My system keeps a still person marked as present in software, and a planned second radar module will detect them directly from their breathing.
- **Cheap and open:** about €40 of sensors, all code written by me and public in my repository — not a subscription cloud product.
- **Live positions, not just "motion yes/no":** the dashboard shows *where* someone is and where they walk, which no PIR or door sensor can do.

What already exists — competitor analysis

Before building anything I researched what already solves this problem. The short answer: **quite a lot, and several products do it better than I will.** I think that is worth stating plainly rather than pretending otherwise.

Full research with sources is in `ai-docs-research/COMPETITORS-`

`*.md`.

Products that solve the same problem

Product	Price	How it compares
Aqara FP2	€55–85	Tracks 5 people, 30 zones, fall detection, and its app already shows alive map with people moving on it — the polished version of my dashboard
Everything Presence Lite	~€33	The same LD2450 radar on the same ESP32, in a proper case. Cheaper than my parts if I count mistakes
Apollo MSR-2	€35–55	ESPHome-based multisensor, but uses the LD2410B — presence only, no x/y positions
Sonoff / Tuya / IKEA sensors	€10–25	Cheap occupancy booleans. No positions, and the IKEA one is PIR so it goes blind when you sit still
Ring / Arlo / Eufy cameras	€40–250 + €4–20/month	More capable, but a camera in the room and mostly cloud storage — the thing I am specifically avoiding
Vayyar Care, Cherish Serenity	€240–300 + €20–39/month	Radar elder-care systems with certified fall detection. A different, more serious product class

The WiFi-sensing part is not new either

The most surprising thing I found: the WiFi-CSI sensing I planned as my "research layer" is **already a shipping commercial product**. Cognitive Systems' WiFi Motion runs in roughly a million homes through 120+ internet providers. Verizon sells it as "Home Awareness". Comcast bundles it free into "Xfinity Shield". The IEEE standardised it as 802.11bf in 2024, and WiFi chips now advertise built-in support for it.

So I am not inventing anything. I am reimplementing a productised technique on €10 of hardware in order to understand how it works.

Usefully, this also confirmed a design decision I had already made: all the commercial WiFi-sensing products detect only **coarse motion**, never precise positions, because the signal patterns depend heavily on each specific room. That is exactly why I use radar for the actual tracking and treat WiFi sensing as an experiment rather than the part I rely on.

So why build it?

Not because the result will be better. Three honest reasons:

1. **Nothing here does exactly this combination.** Every LD2450 project I found assumes Home Assistant. Mine is a standalone application I wrote myself, and no existing project combines radar tracking with a WiFi-sensing experiment sharing one dashboard and coordinate system. (Two hobby projects, [joshuabarraza/mmwave-radar](#) and [PeterkoCZ91/HLK-LD2450-security](#), do come very close on the radar half — the second is ahead of mine, with Kalman filtering.)
2. **No subscription and no cloud.** The commercial ambient-sensing products cost €20–40/month forever. There is also a concrete privacy example: in August 2026 it was reported that Comcast's movement logs can be handed to law enforcement without telling the subscriber, and that switching the feature off does not delete logs already collected. A self-hosted system reachable only through my own VPN avoids that by design, not by policy.
3. **The point is the learning.** Buying an FP2 would give me a working sensor and teach me nothing. Building this means understanding every layer, from how bytes arrive over a serial wire to how a dot gets drawn on a canvas.

Where I do not compete: this is not a security product (no professional monitoring, no certified alarm response) and not a medical device (no fall detection). Those are out of scope.

Possible issues I expect (and some that only look like issues)

Real challenges I will have to deal with:

- **Radar sees through walls.** 24 GHz radio passes through drywall, so the sensor can pick up my neighbor or someone in the hallway and count them as "in my room". I plan to limit the detection range and point the sensor away from shared walls, and filter out positions outside my room in software.
- **False alarms from moving objects.** Fans, swaying curtains and airflow from heating can look like a person to a radar. This is why I add the cheap infrared sensor as a cross-check, and why the validation phase includes an overnight empty-room test.
- **A perfectly still person can disappear.** The tracking radar is built for moving targets; someone sitting frozen can drop off it. For now the server works around this: if a person vanishes in the middle of the room (not near the door), it keeps them marked as present. The planned LD2412 upgrade solves it properly — that module specializes in detecting the micro-movement of breathing.
- **The building's WiFi may block my devices from talking to each other.** Shared/student networks often isolate devices from one another for security. If that happens here, my sensor node connects to the PC with a USB cable instead — less elegant, still works.
- **It can only track 3 people at once,** and when people cross paths the tracker can briefly mix them up. Fine for a studio room, a real limit for bigger use.
- **Remote access must not mean public access.** A dashboard showing whether my room is empty would be useful to a burglar too. It will only be reachable through my private VPN (Tailscale), never the open internet.

And questions people reasonably ask that turn out fine:

- **"Is being radiated by a radar safe?"** Yes — it transmits a few milliwatts, far less than the phone in your pocket, at frequencies also used by car parking sensors and motion-controlled lights.
- **"Does it interfere with WiFi?"** No — it works at 24 GHz, ten times higher than 2.4 GHz WiFi; they don't overlap.
- **"Can it identify me?"** No — it outputs coordinates. It cannot tell who someone is, which is a feature, not a limitation.

Languages, frameworks and tools — and why I chose them

What	Choice	Why this one
Server language	Python 3	industry standard; has the best libraries for working with sensor/signal data (NumPy, SciPy), which I need for the WiFi-sensing experiment. Also the language I'm most familiar with and will meet again in AI Basics.
Server framework	FastAPI + Uvicorn	Modern Python web framework with built-in WebSocket support — the dashboard needs a live stream of positions (10×/second), not page refreshes. Very little boilerplate, so I can understand every line.
Dashboard	Plain HTML/JavaScript/Canvas	No build tools, no framework to learn on top of everything else. One file, opens in any browser (PC and phone), and drawing dots on a Canvas is exactly what a radar view needs. A framework like React would add complexity without adding anything I need.
Sensor node firmware	ESPHome (YAML configuration)	ESPHome has ready, community-tested drivers for my exact radar modules. The alternative — writing C++ myself to parse each radar's serial protocol — means re-solving problems thousands of people already solved, including bugs caused by firmware differences between sensor batches. I connect to it from Python with the <code>aioesphomeapi</code> library.

CSI experiment firmware	Espressif esp-csi (C, flashed as-is)	The chip manufacturer's own example firmware for CSI capture; recommended method in their documentation.
Version control	Git + GitHub	Standard practice; also my backup and the home of this documentation.
Remote access	Tailscale (VPN)	The dashboard must never be on the open internet; Tailscale gives every one of my devices a private encrypted connection with zero server configuration.

Requirements

Functional requirements (what the system must do)

- **FR1** — Detect whether at least one person is present in the room.
- **FR2** — Show the position of each detected person (up to 3) as a dot on a top-down room map, updating at least 5× per second.
- **FR3** — Keep reporting a person as present when they sit or lie completely still (v1: software hold — a person who disappears away from the door stays counted; later: directly, via a second radar that detects breathing).
- **FR4** — Show the current state (empty / occupied / number of people) in text form next to the map.
- **FR5** — Ignore people/movement outside the room boundaries (filter radar detections behind walls).
- **FR6** — Work without any camera or microphone.
- **FR7** — Be viewable from my phone outside my home, and send a notification when presence is detected while the system is "armed".

System requirements (what is needed to run it)

- A PC (any OS) with Python 3.11+ and a modern browser; later a Raspberry Pi Zero W takes over as the always-on host.
- 1× ESP32 dev board for the sensor node; 2 more for the WiFi-sensing experiment.
- HLK-LD2450 radar module (+ optional AM312 PIR; HLK-LD2412 later), jumper wires, breadboard, 5 V USB power.
- A WiFi network the node and the PC can share — or a USB cable as fallback if the building's WiFi isolates devices from each other.
- Phone: any browser (the dashboard is a web page); Tailscale app for remote access later.

Priorities (MoSCoW)

- **Must have:** real radar data on the dashboard (FR1, FR2), state display (FR4).
- **Should have:** still-person hold (FR3), out-of-room filtering (FR5), the written validation tests.
- **Could have** (if time in the block allows): PIR cross-check, the WiFi-CSI live demo, phone notifications, 24/7 Raspberry Pi hosting, automatic arm/disarm, native app, bb or nerf gun turret in a corner that shoots the detected presence inside the room.

SWOT analysis

	Helpful	Harmful
Internal	Strengths: no camera → privacy by physics; cheap (~€30 sensors); shows real positions, not just "motion"; fully open source and documented; research trail with 70+ sources behind every hardware choice	Weaknesses: covers one room only; max 3 people and IDs can swap; I'm a first-semester student learning most of these technologies for the first time; depends on cheap sensors with known batch-to-batch firmware differences

External	Opportunities: presence-without-cameras is a growing real market (elderly care, smart buildings); project extends naturally into later semesters (AI on the CSI data, mobile app); strong portfolio piece	Threats: parts ship from China (delay risk inside a 4-week block); building WiFi may block device-to-device traffic; radar false positives from fans/heating could undermine trust in the demo
-----------------	--	---

Risk analysis

Risk	Chance	Impact	My mitigation
Radar modules arrive late	Medium	High (nothing to demo)	Ordered in week 1; meanwhile I develop against the built-in simulator, so all software progresses without hardware; simulator also serves as backup demo
Building WiFi blocks node→PC traffic (client isolation)	Medium	Medium	Tested with a simple ping test; fallback is a USB cable, which is already the plan for the CSI receiver anyway
False positives (fan, curtains, neighbor through the wall)	High	Medium	Range limiting, aiming away from shared walls, software room-boundary filter, PIR cross-check, overnight empty-room test in validation
Sensor batch has a different serial protocol version	Low	Medium	Using ESPHome drivers that already handle known variants; logging raw data on first boot

Scope too big for 4 weeks	High	Medium	MoSCoW priorities above; "Must have" alone is a complete, demonstrable product
My knowledge gaps block progress	Medium	Medium	Research-first approach (documented), community guides for every component, AI assistant for reviews, teachers for feedback
Losing work	Low	High	Git with pushes to GitHub after every work session

How to get it running (for someone starting from zero)

Current state — software only, using the simulator (no hardware needed):

1. Install Python 3.11+ and Git.
2. `git clone` the repository, then inside it: `pip install -r requirements.txt`
3. Run `python main.py` — this starts the server in simulation mode.
4. Open `http://localhost:8000` in a browser: you see the room map with simulated people moving around. `python main.py --help` lists other modes.

With hardware (once the build phase is done, full guide will follow):

5. Wire the LD2450 to the ESP32 (4 wires: 5V, GND, TX, RX) as shown in the wiring diagram.
6. Flash the ESP32 with the ESPHome configuration file from this repository (one command; ESPHome walks you through the WiFi setup).
7. Start the server with `python main.py --mode esphome` — dots on the map are now real people.

Expected learning outcomes

By the end of this project I expect to be able to:

- build a complete IT product end-to-end: sensors → microcontroller → server → live dashboard, and explain every part of it;
- work with electronics basics: wiring modules, serial (UART) communication, power considerations;
- understand networking concepts hands-on: WiFi, UDP, client isolation, VPNs;
- write and justify requirements, and validate a product against them with a test plan;
- do exploratory research properly: multiple sources, primary documentation, recording why each choice was made;
- reflect on what a beginner can realistically build in 4 weeks, and scope accordingly.

Applied areas of knowledge (Smart Home challenge topics)

- **Intelligent Technologies** — combining two radar types + PIR into one reliable presence picture (sensor fusion); the WiFi-CSI sensing experiment.
- **AI Basics** — the CSI experiment produces labelled data (radar = ground truth) that a later classification model can be trained on; the block itself uses signal processing, the step before AI.
- **Network & Cloud Basics** — WiFi topology, UDP streaming, client isolation, VPN-based remote access.
- **Cyber Security Principles** — privacy-by-design (no images exist), keeping the dashboard off the public internet, thinking about what an attacker learns from a presence system.
- **Software Design & Engineering** — requirements, architecture, MoSCoW prioritisation, version control, testing and validation.
- **Process Data in Organisations** — a live data pipeline: raw sensor frames → cleaned positions → visualisation and (later)

notifications.

Planning beyond the block

The block delivers the core system. Afterwards I want to continue with: phone app (installable web app) with notifications, moving the server to a Raspberry Pi so it runs 24/7, and expanding the WiFi-sensing experiment into a proper comparison against the radar, and potentially adding a bb or nerf gun on a 3D printed mount in a corner of the room so that it shoots the detected presence.